

Final Presentation Script

Total time: ~20 minutes

- Requirements Implemented: 2 min
 - Demo x3: 12 min
 - High-Level Architecture: 4 min
 - System Components: 2 min
-

1. Requirements Implemented (2 min)

This semester our team implemented three main features.

The first is **Allow Failures**. You can mark a job in your pipeline with `failures: true`, which means if that job fails, it won't affect the overall pipeline status. This is useful for non-critical jobs like coverage reports or optional security scans — jobs you want to run but don't want blocking your releases.

The second is **Observability**. We integrated a full observability stack — Prometheus for metrics, Loki for structured logs, and Tempo for distributed traces, all visualized through Grafana. Every pipeline execution automatically produces metrics, logs, and traces, giving you end-to-end visibility into what happened and when.

The third is **Kubernetes and Helm**. The stateless components of our system can be deployed to a Kubernetes cluster using a single Helm chart. Every pipeline job runs as an independent Kubernetes pod, and gets cleaned up automatically when it finishes.

2. Demo 1 — Allow Failures (3 min)

```
Run: ./demo-scripts/demo1-allow-failures.sh
```

Step 1 — verify: Let's start by verifying the pipeline configuration file to make sure it's valid.

Step 2 — dryrun: Now we do a dry run, which previews the job execution order based on the `needs` dependency graph — without actually running anything.

Step 3 — run allow-failure: Now let's run the `allow-failure` pipeline. In the test stage, there's a job called `coverage-report` whose script is just `exit 1` — it will always fail. But it has `failures: true` set.

```
(point at coverage-report failed (allowed) in output)
```

You can see `coverage-report` did fail, but the system logged a warning instead of stopping. The pipeline continued executing, and the final result is Pipeline Completed Successfully. The failure was absorbed.

Step 4 — run block-failure: Now compare that with the `block-failure` pipeline. `critical-test` also runs `exit 1`, but it does NOT have `failures: true`. Watch what happens — as soon as `critical-`

`test` fails, the system stops immediately, the entire release stage is skipped, and the pipeline ends with FAILED. That's the difference between an allowed failure and a blocking failure.

3. Demo 2 — Observability (5 min)

Run: `./demo-scripts/demo2-observability.sh`

Step 1 — run pipeline: Let's run a clean pipeline. This will generate metrics, logs, and traces all at once.

Step 2 — Pipeline Overview dashboard: This is the Pipeline Overview dashboard. It shows run counts broken down by status, duration trends over time, and a recent runs table. This data comes from Prometheus, which scrapes our server's `/metrics` endpoint every 15 seconds.

Step 3 — Stage/Job Breakdown dashboard: The Stage and Job Breakdown dashboard gives you per-stage and per-job duration distributions. You can quickly see which job is the bottleneck across multiple runs.

Step 4 — Logs Viewer dashboard: The Logs Viewer shows structured logs. Every log entry carries context fields like `pipeline`, `run_no`, `stage`, `job`, and `source` — so you can filter down to exactly the logs you care about. These logs flow from our server through the OpenTelemetry Collector into Loki.

Step 5 — Trace Explorer dashboard: Finally, the Trace Explorer shows distributed traces. Each pipeline run is a root span, with every job as a child span underneath it. You can see the exact start time, end time, and duration of each job. The root span carries the `pipeline` name and `run_no` as attributes, which lets you correlate traces with logs and metrics.

4. Demo 3 — Kubernetes & Helm (4 min)

Run: `./demo-scripts/demo3-kubernetes.sh`

Step 1 — helm list: Let's look at our Kubernetes deployment. `helm list` shows we have one release called `cicd`, deployed from our Helm chart. A single `helm install` command brought up the entire system.

Step 2 — kubectl get pods / svc: All pods are running — the CI/CD server, MongoDB, and the full observability stack: Prometheus, Loki, Tempo, OpenTelemetry Collector, and Grafana. Everything in one cluster.

Step 3 — watch pods while running pipeline: Now let's run a pipeline and watch the pods in real time. Notice new pods appearing — each one is a Kubernetes Job pod for an individual pipeline job. The pod runs the job's script inside the specified container image, with a shared PVC mounted at `/workspace` so jobs within the same stage can pass files to each other. Once the job finishes, the pod is deleted automatically.

Step 4 — helm get values: All of the system configuration is managed through `values.yaml` — image versions, replica counts, storage sizes, service types, ports. Users can override any of these at install time with `--set` or a custom values file. This is what configuration-as-code looks like for infrastructure.

5. High-Level Architecture (4 min)

(open README architecture diagram)

Walking through the architecture — the user interacts through the CLI, which sends HTTP requests to the server on port 8080. When a run request comes in, the server hands it off to a thread pool managed by WorkerManager. The RunWorker handles the actual execution: it calls JobScheduler to topologically sort jobs by their **needs** dependencies, then executes jobs concurrently within each dependency level.

In Kubernetes mode, each job becomes a K8s Batch Job object created via the Kubernetes API. In local mode, jobs run as Docker containers. Either way, the execution model is the same from the user's perspective.

In parallel with execution, metrics are recorded to Prometheus via the **/metrics** endpoint, and logs and traces are sent to the OpenTelemetry Collector over gRPC, which fans them out to Loki and Tempo respectively. Grafana sits on top of all three as the unified visualization layer.

6. System Components (2 min)

(open README MongoDB schema diagram and Observability diagram)

On the data side, MongoDB has three collections. **pipeline_runs** stores the overall execution record including the trace ID for correlation. **stage_runs** stores per-stage status and timing. **job_runs** stores per-job details including the **failures** flag. Run numbers are auto-incremented per pipeline using the **counters** collection.

On the observability side, the server produces three types of telemetry simultaneously — metrics exposed at **/metrics**, and logs plus traces sent over OTLP gRPC to the OpenTelemetry Collector. The Collector batches the data and routes logs to Loki and traces to Tempo. All four Grafana dashboards are provisioned as JSON files committed to the repository, so the entire observability setup is reproducible from code with no manual configuration required.

Q&A — Kubernetes & Helm

Q: Why use Helm instead of plain kubectl apply? Helm packages the entire system as a single deployable unit. We have 7 components — server, MongoDB, Prometheus, Loki, Tempo, OTEL Collector, and Grafana. With plain kubectl you'd manage 7 separate manifests and coordinate deployment order manually. With Helm, **helm install cird helm/cird-chart/** deploys everything in one command, with all configuration centralized in **values.yaml**.

Q: Which components are stateless and which are stateful? The server and OTEL Collector are stateless — they hold no local state, so they can be restarted freely. MongoDB, Prometheus, Loki, Tempo, and Grafana are stateful — they all persist data, which we handle with PersistentVolumeClaims.

Q: What RBAC permissions does the server have? We follow least privilege. The server's ServiceAccount has permissions to create/get/list/watch/delete **batch/jobs**, get/list/watch **pods** and **pods/log** (for capturing job output), and create/get/delete **persistentvolumeclaims** for workspace management. No access to secrets or anything else.

Q: How does a pipeline job actually run on Kubernetes? For each job, the server creates a Kubernetes Batch Job object named `cicd-job-{jobLabel}-{timestamp}`. The pod uses the image specified in the pipeline YAML, runs the script as `sh -c "cmd1 && cmd2"` in `/workspace`. After completion, the server captures the pod logs then deletes the Job. BackoffLimit is 0, so there are no retries.

Q: How do jobs in the same stage share files? At the start of each pipeline run, we create a PVC named `workspace-run-{runNo}` (100Mi, ReadWriteOnce). Every job in the run mounts this PVC at `/workspace`. So if a compile job writes a binary to `/workspace/build`, the test job can read it directly. The PVC is deleted after the entire pipeline finishes.

Q: Why is imagePullPolicy set to Never? We're running on minikube with a locally built image that hasn't been pushed to any registry. `imagePullPolicy: Never` tells Kubernetes to use the local image directly. In production you'd push to a registry and use `Always` or `IfNotPresent`.

Q: What happens to a running pipeline if the server crashes? Currently there's no crash recovery. The pipeline status stays as `running` in MongoDB, and the K8s Jobs may still be executing in the cluster, but the server won't track them after restart. This is a known limitation for the current scope.

Q&A — Observability

Q: Walk me through the observability stack design. We have three signal types: metrics via Prometheus pull, logs and traces via OpenTelemetry push. The server uses the OTEL Java SDK to emit logs and traces to the OpenTelemetry Collector on port 4317 over gRPC. The Collector batches and routes logs to Loki and traces to Tempo. Separately, Prometheus scrapes `/metrics` every 15 seconds. Grafana connects to all three as datasources and visualizes everything in four dashboards.

Q: What metrics do you expose? Five metrics total:

- `cicd_pipeline_runs_total` — Counter, labels: pipeline, status
- `cicd_pipeline_duration_seconds` — Histogram, label: pipeline, buckets up to 600s
- `cicd_stage_duration_seconds` — Histogram, labels: pipeline, stage, buckets up to 300s
- `cicd_job_duration_seconds` — Histogram, labels: pipeline, stage, job, buckets from 0.5s to 120s
- `cicd_job_runs_total` — Counter, labels: pipeline, stage, job, status

The histogram bucket sizes decrease at each level — job-level is finer-grained, pipeline-level handles longer durations.

Q: What does a trace look like for a pipeline run? Each pipeline run creates a root span named `pipeline:{name}` with attributes `pipeline` and `run_no`. Every job creates a child span named `job:{name}` under that root span. In Trace Explorer you see the full hierarchy with precise start time, end time, and duration for each job. The trace ID is stored in MongoDB, so you can retrieve it from the `report` command and use it to look up the trace directly.

Q: What fields are in your structured logs? Logs are JSON format via Logstash encoder. Beyond the standard timestamp, level, and message, we inject context via MDC: `pipeline`, `run_no`, `stage`, `job`, `source` (either `system` for server logs or `container` for job stdout/stderr), `status`, `duration_sec`, `branch`, `commit`, and `trace_id`. Every log entry is fully queryable by any of these fields in Loki.

Q: How do you collect stdout/stderr from job containers? In Docker mode, DockerJobExecutor attaches to the container's exec output in real time, logging each line with `source: container` in MDC. In Kubernetes mode, KubernetesJobExecutor calls `readNamespacedPodLog` after the Job completes, then logs each line the same way. Both paths send the logs through the OTEL Appender into Loki.

Q: Why manage Grafana dashboards as code? The requirement explicitly states that UI-created dashboards don't satisfy the requirement because they aren't reproducible. Our dashboards are JSON files in `observability/grafana/dashboards/`, provisioned automatically at Grafana startup via `dashboard-provider.yaml`. Any new environment gets identical dashboards without any manual setup.

Q: What is the OpenTelemetry Collector doing exactly? Why not send directly to Loki and Tempo? The Collector decouples the server from the backend storage. The server only knows about one endpoint (the Collector at port 4317), regardless of what's behind it. The Collector applies two processors: a batch processor (5s timeout, 1024 batch size) to reduce network overhead, and a resource processor that stamps every signal with `service.name: cicd-server`. If we ever want to swap Loki for Elasticsearch or add a second backend, we change only the Collector config — not the server code.

Q: How are logs and traces correlated in Grafana? The Loki datasource is configured with a Derived Field that uses the regex `"traceId": "(\\w+)"` to extract the trace ID from log entries. This creates a clickable link in the Logs Viewer that jumps directly to the corresponding trace in Tempo — no manual copy-pasting required.

Q: What are your data retention policies? Prometheus retains 7 days via `--storage.tsdb.retention.time=7d`. Loki retains 168 hours (7 days) configured in `loki-config.yaml` with the compactor handling cleanup. Tempo uses local filesystem storage without a retention limit, which is acceptable for our demo scope. All three align at 7 days so historical data can be correlated across signals.

Q: What's the difference between local and Kubernetes observability setup? Functionally identical. The only difference is service addressing: in Docker Compose, Prometheus targets `server:8080` and the OTEL Collector routes to `loki:3100` and `tempo:4317`. In Kubernetes, these become `{release-name}-cicd-server-service:8080`, `{release-name}-loki-service:3100`, and so on. The Helm templates handle this automatically using `{{ .Release.Name }}` — no manual changes needed between environments.